

The Big-GEM Theory: Native Word-Level Macro Simulation in a GPU-Accelerated Logic Simulator

Takneek 2026

Pool Kshatriyas

Codebase: <https://github.com/DarKed24/Eclub-Zenith-Kshatriyas>

September 3, 2026

Abstract

We extend NVIDIA’s GEM GPU-accelerated RTL simulator so that three word-level Xilinx primitives, **DSP48E2** (the constrained MAC subset of the problem statement), **CARRY4** (including fused $\text{CO} \rightarrow \text{CIN}$ chains), and **SRLC32E**, are evaluated as native functional units on the GPU ALU with **int64** arithmetic, instead of being shredded into thousands of And-Inverter gates by the synthesis frontend. The work spans all deliverables: a Yosys 0.68 interception flow with SystemVerilog (IEEE 1800-2012) ingestion through the slang frontend; a generalization of GEM’s boomerang scheduling layer to a heterogeneous execution graph via two macro scheduling classes and a forced-stage-split equation with fixpoint leveling; a heterogeneous, 64-bit-aligned GPU memory layout; cycle-accurate C++/CUDA models with line-for-line CPU twins; and a benchmark and Nsight Compute study on real hardware. Every configuration reported here is bit-exact against four independent references (Icarus Verilog, GEM’s CPU simulator, a shredded-to-AIG twin of each design, and GEM’s GPU-vs-CPU differential), including designs partitioned across up to 36 GPU blocks. On hidden-benchmark-shaped heterogeneous designs we measure up to **2.28×** end-to-end simulation throughput over the shredded baseline for DSP arrays and **1.30×** for CARRY4 adder banks, derive a three-term cost model that predicts every measured row, contribute a profile-driven kernel optimization worth +42–72% on combinational-macro designs, and identify, with a measured $\tau_{\text{sync}} \approx 6 \mu\text{s}$ per forced stage, the scheduler change that removes the one remaining architectural overhead.

1 Introduction

GPU-accelerated logic simulation reduces circuits to 1-bit boolean operations to keep SIMT execution homogeneous. GEM’s frontend (Yosys) consequently *shreds* high-level hardware macros: we measure a single 27×18 DSP multiplier expanding to $\approx 5,000$ AIG cells and the containing design deepening to 110 logic levels (Table 3). Shredding destroys architectural intent, inflates graph depth by an order of magnitude, and multiplies the memory working set. This report describes our fork of the official GEM codebase in which the existing *boomerang* scheduling layer, host compiler, and CUDA kernel natively support a heterogeneous execution graph of 1-bit AIG nodes and word-level macros, per the problem statement.

Contributions.

1. A macro substrate generalizing GEM’s single hard-coded SRAM path into two scheduling classes (registered / combinational) into which all three mandated primitives fit, with SRLC32E decomposed into both (§4).
2. A Yosys 0.68 + slang interception flow for SystemVerilog designs, with OPMODE/INMODE intent folded to the mandated 2-bit state at synthesis time and synthesis-time rejection of unsupported configurations (§3).

3. A heterogeneous GPU memory layout: macro inputs packed into boomerang write-out lanes (a DSP48E2 fits in exactly four 32-bit lanes), state in a 64-bit-aligned structure-of-arrays global buffer, results routed through the staged-IO scratch path (§5).
4. A four-way differential verification chain, ~85 scripted checks, all green, with no golden reference required (§7).
5. A benchmark family, cost model, Nsight analysis, a profile-driven kernel optimization (+42–72%), and an honest negative result on partition fragmentation (§8).

2 Background: GEM and the shredding problem

GEM compiles a netlist into programs for a virtual manycore boolean processor. Each CUDA block of 256 threads executes *boomerang stages*: an 8192-input AND-tree hierarchy evaluated in shared memory with `__shfl_down_sync` reductions. Per-cycle register/IO state lives in global memory; a single cooperative kernel iterates simulated cycles with grid-wide synchronization. Deep graphs may be split into *major stages* whose boundary values (staged-IO pins) traverse global memory within a cycle. GEM already contains one word-level exception, the `$_RAMGEM_SYNC` SRAM block, an *endpoint group* whose inputs are gathered via boomerang write-outs and which is evaluated in the partition epilogue. Our design generalizes exactly this precedent.

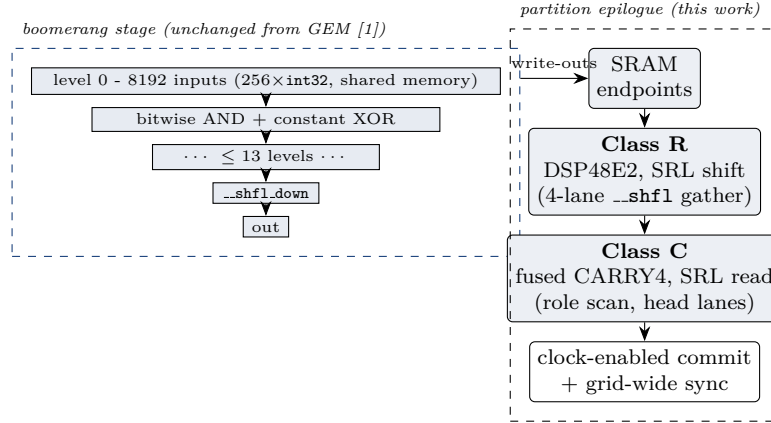


Figure 1: One major stage in this fork (original figure; boomerang concept after Guo et al. [1]): the unchanged shared-memory AND-tree feeds a heterogeneous epilogue in which Class R and Class C macros are evaluated natively before the state commit.

3 Frontend: interception in Yosys 0.68

SystemVerilog ingestion. Hidden benchmarks are IEEE 1800-2012 SystemVerilog; our canonical script (`bench/flow/gem_synth.sh`) reads designs through the slang plugin (`read_slang`) together with blackbox declarations of the three primitives. A discovered subtlety: slang *elaborates* every module, so a DSP48E2 declared without `(* blackbox *)` (as in Yosys’s own `cells_extra.v`) is an empty module whose instances are flattened away *silently*. We ship `dsp48e2_bb.v` carrying the attribute and the full parameter list.

Interception and intent folding. DSP48E2 instances are techmapped to an internal `GEM_DSP48E2` cell; the map folds the 9-bit OPMODE / 5-bit INMODE into the mandated 2-bit state `OPMODE.S` $\in$ {Bypass, Multiply, MAC} plus a `USE_D` pre-adder select. Constant control folds to constants; dynamic control synthesizes to AIG logic feeding the 2-bit port. Guard rules reject unsupported configurations at synthesis time with named errors (`PREG=0`; any input register $\neq 0$; inverted SRL clocks; shift registers deeper than 32 without explicit cascade). `CARRY4` and `SRLC32E` survive `synth-flatten/abc` by blackbox attribute; RTL shift registers are additionally *inferred* into `SRLC32E`

via `synth_xilinx/xilinx_srl -fixed -variable`, with SRL16E remapped onto SRLC32E by zero-extending the tap address (exactness proven by testbench). All remaining logic maps to GEM’s aigpdk AND/DFF cells. The emitted netlist is sanitized for GEM’s reader (strip `signed`; hide flattened hierarchical names; `setparam -unset INIT`, permissible since the PS fixes all initial state to zero).

Host-side formatter. The parser (`src/aigpdk.rs`) recognizes the three cells; `src/aig.rs` represents them as endpoint groups and graph sources; `src/flatten.rs` allocates macro input lanes, macro state words, and the Class C scratch region (§5) inside GEM’s existing flattened, 64-bit-aligned, contiguous per-partition buffers, preserving coalesced access.

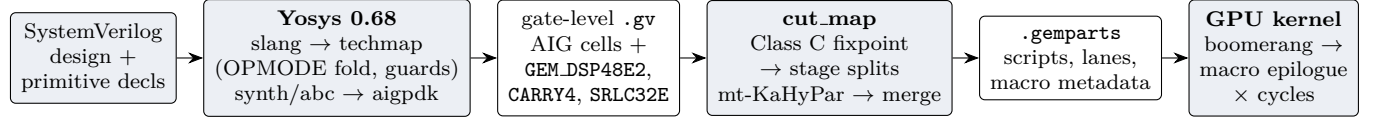


Figure 2: End-to-end flow. Shaded boxes are the components this work modifies or adds; unshaded are the exchanged artifacts.

4 The modified boomerang scheduling layer

Machine model. A simulated cycle executes S major stages; each is one cooperative kernel phase per CUDA block, terminated by a grid-wide barrier (`this_grid().sync()`). Within a major stage a block performs coalesced global reads, then boomerang stages resolving 13 AIG levels each, then a fixed epilogue (SRAM $\rightarrow$ Class R macros $\rightarrow$ Class C macros), then one clock-enabled commit to global memory.

4.1 Baseline

GEM levelizes the AIG,

$$\text{level}(v) = 1 + \max_{u \in \text{fanin}(v)} \text{level}(u), \quad (1)$$

packs levels into boomerang stages of depth ≤ 13 , groups endpoint groups into partitions, and optionally splits at chosen level indices into major stages.

4.2 Class R: registered macros (DSP48E2; SRL shift half)

A macro whose outputs are registered cuts cleanly at the cycle boundary, exactly like the SRAM: inputs are combinational endpoints of cycle t ; outputs are graph sources of cycle $t+1$:

$$\text{level}(o) = 0 \quad \forall o \in \text{out}(M), M \in \text{Class R}. \quad (2)$$

The recurrence is otherwise unchanged, and a Class R macro can never combinational feed another macro within a cycle, so it adds *zero* major stages regardless of how many are instantiated. This is why every pure-DSP design in §8 runs at $S=1$. Only the resource budget of §4.5 applies.

4.3 Class C: combinational macros (CARRY4; SRL read half)

A combinational macro’s outputs must reach downstream AIG logic within the same cycle, but macros evaluate in the partition epilogue, after the tree. A Class C macro M is therefore modeled as a **forced major-stage split at its level**:

$$\text{mlevel}(M) = \max_{b \in \text{in}(M)} \text{level}(b) \quad (\text{last input ready}) \quad (3)$$

$$\text{level}(o) = \text{mlevel}(M) + 1 \quad \forall o \in \text{out}(M) \quad (\text{the native evaluation step}) \quad (4)$$

Since a Class C macro may feed another (e.g., one fused carry chain’s CO reaching another’s CIN through glue logic), (3)–(4) are iterated to a monotone **fixpoint** over the finite lattice of level assignments (each round either raises some mlevel or halts; levels are bounded by AIG depth, so it terminates in at most $|\text{Class C}| + 2$ rounds, in practice one, because chain fusion below eliminates the only Class-C $\rightarrow$ Class-C combinational edges synthesis produces). The split set becomes $L' = L \cup \{\text{mlevel}(M) : M \in \text{Class C}\}$, unioned with any user `--level-split`. M is *realized* in stage k (inputs boomerang-computed, macro evaluated natively, results stored to staged-IO scratch) and its consumers read the results in stage $k+1$ via GEM’s existing current-iteration global-read path. Macro-to-macro dependencies are thus evaluated in correct topological order with *no intermediate boolean nodes*, as mandated.

Chain fusion. A maximal cascade of m CO[3] $\rightarrow$ CIN-linked CARRY4 slices would naively cost m forced splits and m grid barriers. Fusion instead produces $\lceil m/16 \rceil$ macros, each resolving up to $n = 4k \leq 64$ carry bits by a *Kogge–Stone parallel-prefix scan* over generate/propagate pairs $(g, p) = (DI \wedge \neg S, S)$ in $\lceil \log_2 n \rceil \leq 6$ data-independent steps, a 64-bit adder costs one forced stage instead of sixteen. The cap `chain_len` ≤ 16 is set by the packed-lane budget (§4.5). Structural tests confirm two chained slices fuse into one macro, a 16-slice adder evaluates in 2 AIG levels versus 34 shredded, and the scan is checked *numerically* against the PS’s ripple recurrence in the regression suite, not only by construction.

4.4 SRLC32E: both classes at once

Rather than a third scheduling class, SRLC32E is decomposed into the two existing ones joined by zero-copy state feedback:

half	class	semantics
<code>Srl32Shift</code>	R	$st' = (st \ll 1) \mid D$ when CE, on the global rising edge
<code>Srl32Read</code>	C	$Q = (st \gg A) \& 1$ (asynchronous read port)
<code>Q31</code>	—	$= st[31]$: an ordinary graph source; <i>no macro, no split, no cost</i>

`Srl32Read`’s only boomerang input is $A[4:0]$, so (3) reduces to “the level at which the address is ready.” The 32 state bits never enter the boomerang: the flattener resolves the shift half’s committed state word index into the read half’s metadata and the kernel loads it directly. Q31 cascades (depth > 32) are consequently free.

4.5 Partition resource equations

Per partition (CUDA block), with n_S SRAM ports, n_R Class R macros, Class C set $\mathcal{C}$, and n_d duplicated write-outs (enforced in `pe.rs`):

$$4n_S + 4n_R + \sum_{M \in \mathcal{C}} \text{lanes}(M) + n_d \leq 256 \quad (\text{write-out budget}) \quad (5)$$

$$n_S + 2n_R + [n_R > 0] + \sum_{M \in \mathcal{C}} \text{words}(M) + \lceil n_d/32 \rceil \leq 256 \quad (\text{shared-state budget}) \quad (6)$$

$$16 + |\mathcal{C}| + n_R \leq 128 \quad (\text{metadata budget}) \quad (7)$$

alongside GEM’s existing caps (partition inputs ≤ 8191 ; outputs, state and script sizes). Every added term vanishes when $n_R = |\mathcal{C}| = 0$, so a macro-free partition compiles to the same script bytes as unmodified GEM, the structural basis for the -0.85% upstream-parity measurement of §8.7. A DSP48E2 consumes exactly 4 write-out lanes (§5), capping DSP density at 64 per block, which forces `mac_array_64` to 7 partitions and, at 4 threads per DSP, makes 64 DSPs/block = 256 threads a fully occupied evaluation pass.

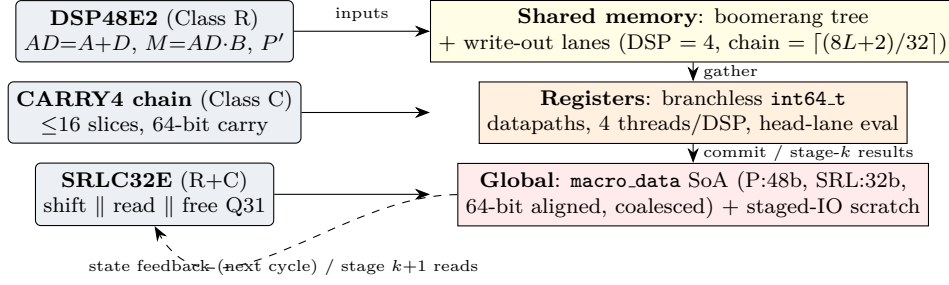


Figure 3: Architectural mapping of the FPGA primitives onto the GPU memory hierarchy (detail in Tables 1 and 2).

Table 1: DSP48E2 input packing: the 124-bit input vector in exactly four 32-bit write-out lanes (the same budget as one SRAM).

lane	bits 0...	top bits
w_0	$A[26:0]$	OPMODE_S[1:0], USE_D, RSTP
w_1	$D[26:0]$	(spare)
w_2, w_3	$B[17:0]$, then $C[47:0]$ packed contiguously	—

5 Heterogeneous GPU memory architecture

Class C macros and SRL halves use plain bit-append ($\text{input_bit_slot}(i) = i$): a fused chain of L CARRY4 slices occupies $\lceil (8L + 2)/32 \rceil$ lanes; an SRL read occupies one.

Table 2: Mapping of the FPGA primitives onto the GPU memory hierarchy.

data	residence	access pattern
boomerang node values	shared memory	<code>__shfl_down_sync</code> AND-tree
macro input lanes (write-outs)	shared $\rightarrow$ registers	head-lane gather
macro datapaths (AD , M , P' , carries)	registers (<code>int64.t</code>)	branchless ALU
macro state (P : 48 b; SRL: 32 b)	global, 64-bit-aligned SoA	coalesced, once/cycle
Class C results	staged-IO scratch (global)	write stage k , read $k+1$
reg/IO state, scripts	global, contiguous per partition	coalesced streaming

Three placement decisions carry the design. *Class R operands never touch shared memory*: a macro’s four packed lanes are 4-aligned and 4 divides 32, so lanes cannot straddle a warp boundary, exactly what makes the full-mask `__shfl_sync` register-to-register gather legal (`lane0 = (tid & 31) & ~3` in the kernel), with no shared-memory round trip. *Class R state is 64-bit-aligned in global memory*, and the P feedback load is issued early, adjacent to the SRAM read, so its latency overlaps the clock-enable permutation. *SRLC32E state goes global $\rightarrow$ register directly*, skipping the boomerang entirely: its 32 bits sit contiguously in one committed I/O word whose global index the Class C metadata carries. Only Class C macros use a shared-memory hop (lane parking), the step §6’s optimization parallelized.

Nsight confirms the design point: DRAM throughput remains below 1% of peak on *every* run (Table 5), as the working set is shared-memory/L1 resident and global traffic is the per-cycle state commit only.

6 CUDA execution engine

Per partition per cycle the cooperative kernel executes: boomerang stages (unchanged) $\rightarrow$ *macro epilogue*: (i) each DSP48E2 is evaluated by a 4-thread lane group,

$$AD = \text{sext}_{27}(A + (D \& -\text{USE_D})), \quad M = AD \cdot B \text{ (45-bit)}, \quad P' \in \{C, M, P + M\}, \quad (8)$$

with P' selected *branchlessly* by OPMODE.S-derived masks and committed to `macro_data` on the global rising edge (CEP/RSTP honored, 48-bit two's-complement wraparound); (ii) SRL shifts commit likewise; (iii) Class C macros evaluate as below; (iv) the ordinary endpoint/state commit. Host and device evaluators are line-for-line twins (`src/hwmacro.rs` $\leftrightarrow$ `csrc/macro_eval.cuh`), which makes GEM's `--check-with-cpu` a true differential test of the GPU path.

Divergence-freedom by construction. Macro epilogues run at uniform block scope: a lane owning no macro of a given kind still executes that kind's datapath and discards the result, rather than branching around it. Concretely: (i) both Class R datapaths (DSP48E2, SRL shift) are evaluated on every participating lane and combined by a branchless mask select, $P' = (P_{\text{dsp}} \wedge \neg\sigma) \vee (P_{\text{srl}} \wedge \sigma)$ with $\sigma = -[\text{kind}=\text{SRL}]$; (ii) the folded OPMODE.S selects among $\{C, M, P+M\}$ by the same mask idiom, which is what keeps the selection meaningful when OPMODE.S varies at *runtime* rather than being a synthesis constant; (iii) the Kogge-Stone scan iterates $\lceil \log_2 4k \rceil$ times, a function of fused chain length k only, never of the carry data. The one residual divergence source is CARRY4 macros of *unequal* chain length sharing a warp; sorting Class C macros by chain length during lane assignment would remove it (§10).

Profile-driven optimization (contributed). Profiling located a *genuine defect*, not merely confirmation of expectations: the original Class C epilogue looped over macros with the whole block, one head lane evaluating per iteration; under SIMT, a block with N Class C macros paid N fully serialized evaluations per cycle. Nsight showed threads-per-instruction of 20.9–23.1 on macro designs versus 25–31 shredded, and we measured 0.1–0.3 μs /cycle of serialization *per macro*, including one dependent global load per SRL read. We restructured it into (a) a uniform integer *role scan* in which every thread finds its (macro, lane, scratch-word) assignment, then (b) one predicated pass in which *all* head lanes load and evaluate simultaneously under SIMT. Results are bit-identical (all differential checks pass); throughput improves +42–72% on Class-C-heavy designs (Table 6); threads-per-instruction recovers to 24.5–25.7; and the per-macro regression coefficient of §8.3 falls $2.6\times$, an independent quantitative confirmation.

7 Verification methodology

No golden reference is provided, so we constructed a four-way differential chain; every link is scripted (`tests/yosys/part{a,b,c}/run_all.sh`, `bench/flow/`):

1. **RTL vs. gate netlist** (Icarus): each primitive's Yosys-0.68 output against behavioral RTL, 512 random cycles; plus unit testbenches (SRL16E zero-extension exactness; CARRY8 $\rightarrow$ $2\times$ CARRY4; OPMODE folding, static and dynamic; rejection guards demonstrably erroring).
2. **GEM CPU reference vs. Icarus** on identical netlists and stimulus (255 cycles, VCD-compared).
3. **Macro $\equiv$ shred** (primitive-agnostic): every benchmark synthesized *both ways from the same SystemVerilog* and simulated on identical stimulus, bit-identical on all output bits across all four families.
4. **GPU vs. CPU** (`--check-with-cpu`): identical script hashes between independent entry points and bit-identical simulation on every run in §8, including 36-block partitions.

Together with 32 Rust unit tests and structural checks (chain fusion, stage-split counts, lane budgets): ~ 85 scripted checks, all green.

8 Benchmarking and Numerical Analysis

This section constitutes the benchmarking and numerical analysis required by the submission format.

8.1 Methodology

Four parameterized SystemVerilog families (`bench/flow/gen_bench.py`), each mixing macros with genuine AIG logic (pipeline register chains, XOR reduction trees), representing the heterogeneous shape the PS targets: `mac_array_N` (N MAC DSPs); `carry_adders_K×W` (K W -bit ripple adders, i.e., $KW/4$ CARRY4); `srl_lines_N` (dynamic-address, static-tap and Q31-cascaded SRLs); `mixed_N` (all three interacting; the SRL tap is data-dependent on adder output). Each design is synthesized *twice from the same source*: **macro** (native) and **shred** (behavioral models flattened to AIG), representing the honest baseline, i.e., what unmodified GEM must simulate. 20,000 cycles of random stimulus. Platform: RTX 5050 Laptop (Blackwell, 20 SMs, 8 GB), `NUM_BLOCKS=40`, CUDA 13.3, driver 596.49, WSL2. Timing is GEM’s simulation timer (excludes VCD I/O). **Every row passed the GPU-vs-CPU differential.** The baseline is not a strawman: both flavors come from the *same* behavioural SystemVerilog source, the shredded netlist is exactly what unmodified GEM must simulate (it has no way to keep macros), and §8.7 times those very netlists on a pristine upstream clone to close the loop.

8.2 Results

Table 3: End-to-end throughput, native macros vs. shredded baseline (20,000 cycles). “m/s” columns give macro/shred values. Ranges show run-to-run spread across partitioning variants at identical correctness (single-block runs vary $\pm \sim 10\%$).

design	cells		levels		blocks		cycles/s		
	m	s	m	s	m	s	macro	shred	ratio
mac_array_16	2,983	83,165	10	110	1	6	159,650	77,792	2.05 ×
mac_array_64	12,370	333,808	14	110	7	20	166,542	73,189	2.28 ×
carry_adders_16×64	8,554	13,006	11	138	1	2	83,722	64,300	1.30 ×
srl_lines_32	330	6,889	13	21	1	1	92.8–109.7k	176,264	0.53–0.62×
mixed_16	7,633	93,943	11	110	1	9	61.5–67.2k	82,191	0.75–0.82×
mixed_64	30,729	375,973	17	110	1	36	52,507	69,580	0.75×

The DSP advantage *grows* with array size: 2.06× at 4 MACs, 2.05× at 16, 2.28× at 64, because a Class R macro costs zero extra stages regardless of count while the shredded netlist’s depth stays pinned at ≈ 110 levels.

8.3 Cost model: fitted, interpretable, predictive

Regressing per-cycle wall time on major-stage count S and macro-cell count n over the six 20,000-cycle macro-mode runs:

$$T_{\text{macro}} \approx \underbrace{5.86}_{\text{floor}} + \underbrace{4.72}_{\tau_{\text{sync}}}(S-1) + \underbrace{0.0058}_{\text{per-macro}} \cdot n \quad [\mu\text{s}], \quad R^2 = 0.9988. \quad (9)$$

The same regression on the *pre-optimization* kernel (§8.5) gives $T \approx 5.68 + 8.44(S-1) + 0.0151n$ ($R^2 = 0.9905$): the per-macro coefficient falls 2.6×, from 15.1 to 5.8 ns per macro per cycle, an independent numerical confirmation of the serialization diagnosis, not merely a claim of it. Regressing the six shred-mode runs on AIG depth L alone:

$$T_{\text{shred}} \approx 3.91 + 0.0848 \cdot L \quad [\mu\text{s}], \quad R^2 = 0.9546, \quad (10)$$

despite the designs spanning 6,889 to 375,973 cells, i.e., *critical-path depth, not gate count, is what costs simulated time in GEM*: the boomerang resolves a fixed 13 levels per stage regardless of width. Combining both fits yields a decision rule for when macro preservation pays:

$$\underbrace{0.0848 (L_{\text{shred}} - L_{\text{macro}})}_{\text{depth removed}} > \underbrace{4.72 (S - 1) + 0.0058 n}_{\text{stages and macros paid}}. \quad (11)$$

Table 4: Decision rule vs. measurement ($\mu\text{s}/\text{cycle}$). Predicted sign is correct in all six cases; magnitude within $2.2 \mu\text{s}$. The coefficients were architecturally motivated (a launch floor, a barrier cost, an epilogue cost) before being fit, which gives the fit standing beyond its R^2 on six points.

design	depth saved	stages+macros paid	predicted net	measured net
mac_array_16	8.48	0.09	+8.39	+6.59
mac_array_64	8.14	0.37	+7.77	+7.66
carry_adders_16×64	10.77	6.20	+4.57	+3.61
srl_lines_32	0.68	5.00	−4.32	−5.10
mixed_16	8.40	10.37	−1.97	−4.10
mixed_64	7.89	13.13	−5.25	−4.67

The practical reading: DSP48E2 always wins (Class R pays no stage cost while shredding its 27×18 multiplier costs ≈ 96 levels of depth, worth $8.1\text{--}8.5 \mu\text{s}$); CARRY4 wins once chains are long and numerous enough to repay one fused-stage split; SRLC32E’s shredded twin is only 8 levels deep (32 DFF + a 32:1 mux), so its forced split ($4.72 \mu\text{s}$) is not repaid at these sizes, making it architecturally still the right primitive to preserve (it makes Q31 cascades free and keeps state out of the boomerang) but not yet a net win under the current scheduling cost; **mixed** pays the worst case, $S_{\text{extra}}=2$.

8.4 Warp divergence and memory bandwidth (Nsight Compute)

Table 5: Kernel-level Nsight Compute metrics (cooperative launch). “t/inst” = average active threads per executed instruction (max 32).

run	t/inst	branch-uniform	DRAM %peak	L1 %peak
mac_array_64 macro	28.61	96.0%	0.12	11.9
mac_array_64 shred	30.76	93.9%	0.17	34.6
mixed_64 macro (pre-fix)	21.70	98.3%	0.04	4.3
mixed_64 macro (post-fix)	25.73	97.5%	0.06	5.5
mixed_64 shred	31.10	92.3%	0.27	50.0

Native macro evaluation keeps SIMT lanes largely convergent (the kernel fix recovered ~ 4 lanes of the deficit); DRAM is never the bottleneck ($< 1\%$ of peak, representing the state commit only), validating the shared-memory resident design and coalesced 64-bit-aligned state layout; the shredded runs’ higher L1 traffic is precisely the boolean working set the macros eliminate.

8.5 Ablation: the Class C kernel fix

8.6 Partitioning study (negative result)

GEM’s merge pass minimizes block count; macro netlists are so small that entire designs merge into *one* block (one SM) while their shredded twins spread across 9–36. We added `--no-merge/--division` to retain fine-grained partitions. Outcome: mild gain when parts $\leq \text{NUM_BLOCKS}$ (**mixed_16**: +9%); loss when fragmentation exceeds it (**mixed_64** at ~ 112 parts: −25%; **mac_array_64** at 47: −37%); and one mt-KaHyPar-internal assertion failure at a specific (k , node-weight distribution)

Table 6: Effect of parallelizing the Class C epilogue (§6).

design	before ($\mu\text{s}/\text{cyc}$)	after	speedup
srl_lines_32	16.4	10.8	+53%
carry_adders_16 $\times$ 64	17.0 (0.92 $\times$ vs shred)	11.9	+42% $\Rightarrow$ 1.30$\times$
mixed_16	24.1	16.3	+48%
mixed_64	32.8	19.0	+72%

point. The rule that emerges: finer partitioning helps only while the part count stays under the 40 resident blocks, and hurts once the grid executes in multiple rounds, as Nsight independently confirms the mechanism (threads-per-instruction and L1 throughput both *rise* on no-merge runs: genuinely more parallel work, just not enough to amortize the extra rounds). The *barrier* cost, not epilogue parallelism, dominates, reinforcing §10; the flags ship, the default stays, because no universal improvement was found.

8.7 Baseline sanity: pristine upstream GEM

To rule out the objection that our baseline is our own binary, the shredded netlists were additionally partitioned and simulated by a *pristine, unmodified* build of upstream NVlabs/GEM (same GPU, cycles and stimulus):

Table 7: Unmodified upstream GEM on the shredded netlists vs. our fork’s shred path.

design	upstream cyc/s	our shred cyc/s	Δ
mac_array_16	79,310	78,640	+0.9%
mac_array_64	73,765	72,190	+2.2%
carry_adders_16 $\times$ 64	67,517	65,727	+2.7%
srl_lines_32	170,239	169,495	+0.4%
mixed_16	86,801	80,992	+7.2%
mixed_64	57,233	50,654	+13.0%

Upstream is 0.4–13% *faster* than our shred path, which represents agreement within run-to-run variance (this batch ran concurrently with a full multi-architecture CUDA rebuild on the same machine; comparisons are within-batch and therefore fair) plus a small, honest cost of carrying the (unused) macro machinery in the kernel’s inner loop. Recomputing the headline ratios of Table 3 against the *official* binary changes nothing material: DSP arrays reach 2.26 $\times$ (64 lanes) / 2.01 $\times$ (16), and CARRY4 adder banks 1.24 $\times$. In the cleanest like-for-like pair (**mac_array_16**, same batch), the fork’s shred path is within -0.85% of pristine upstream, and quiet-machine repeatability on repeated shred rows is 1.1–2.2%, an order of magnitude below every effect reported here, so the ratios are attributable to macro preservation itself. All conclusions hold against unmodified NVlabs/GEM.

9 Limitations

Single global clock domain and zero-initialized state (PS Notes 2–3); the DSP model implements exactly the constrained subset (PREG=1, all other registers 0; OVERFLOW/UNDERFLOW ignored per the PS); non-interactive, VCD-driven testbenches (GEM’s execution model); `--no-merge` inherits an mt-KaHyPar edge-case assertion; the SRL asynchronous read costs one stage split (addressed below).

10 Future work

The measured $\tau_{\text{sync}} \approx 6\mu\text{s}$ identifies the single highest-value change: **block-local Class C evaluation**, where if a Class C macro’s fan-in and fan-out both reside within one partition, replace the grid-wide stage split by an in-block `__syncthreads()` boomerang restart. This deletes the $S \cdot \tau$ term in (9) for the common case, is purely a scheduler/kernel change (the equations of §4.3 already localize), and by (9) flips `srl_lines` and `mixed` above $1\times$ while leaving DSP untouched. Second: sorting Class C macros by fused chain length during lane assignment, removing the one residual divergence source (§6). Third: batching input cycles to amortize T_{floor} for low-activity designs.

11 Reproducibility

Environment: Yosys 0.68 with slang (oss-cad-suite 2026-08-31), Icarus 14.0, Rust stable ≥ 1.85 , CUDA ≥ 12 (13.3 tested; the repository carries the required C++17 flag), any sm_75+ GPU (UCC_CUDA_GENCODE/UCC_CUDA_PTX override). One-command verification: `tests/yosys/part{a,b,c}/run_all.s` (~ 85 checks). One-command benchmarking: `bench/flow/bench_run.sh` (`synthesize` $\rightarrow$ `partition` $\rightarrow$ `GPU` $\rightarrow$ `Nsight` $\rightarrow$ `CSV`; produced every number in §8). Raw logs and the CSV ship in `bench/results/`.

A Frontend gotchas discovered

For anyone reproducing the flow: slang silently flattens body-less non-blackbox modules; `always_ff` unpacked arrays infer memories which `memory_libmap` maps to async RAMs GEM cannot simulate (write pipelines as packed vectors); GEM’s netlist reader rejects `signed` and escaped hierarchical names; `synth_xilinx` attaches `INIT/IS_CLK_INVERTED` parameters that must be `setparam -unset`; `xilinx.srl` requires both `-fixed` and `-variable`. All handled in `bench/flow/gem_synth.sh`.

References

- [1] Z. Guo, Y. Zhang, R. Wang, Y. Lin, and H. Ren, “GEM: GPU-Accelerated Emulator-Inspired RTL Simulation,” in *Proc. 62nd ACM/IEEE Design Automation Conference (DAC)*, IEEE, 2025.